Quest 10: The Phantasmagorithm

Lord of the Wings · ghosts that bounce off walls and each other

> This guide uses the official puzzle text. The story is summarised and the rules are restated in plain words. It explains what the quest asks and the logic behind my solutions, not the code itself. Every number in the examples and traces was checked by running my solutions on the sample data (plus small instrumented copies to print intermediate values).

THE QUESTION

Ghosts move one square per round on a cave map, turning clockwise at walls and turning around when they end a round on the same square; count their collisions.

INPUT

A map of `#` walls, `.` floor and ghosts `>` `<` `^` `v`.

OUTPUT

Part I: the total number of collisions in 10 rounds. Parts II and III: the round in which the last ghost disappears.

WHAT CHANGES BETWEEN PARTS

Part I just counts for 10 rounds. Part II gives every ghost a collision limit (10000) and removes it when it reaches the limit. Part III uses the same rules with a limit of 10^{12}, so you can't simulate every round any more.

The story

While the Ents attack Nulldor from the front as a distraction, the team slips into old caves that lead under the fortress. The tunnels are guarded by Nullghosts, which drift around, turn when they hit rock and bounce off each other. Every collision makes a ghost fade a little, so the team first counts the collisions and then works out when the last ghost in each chamber will have faded away.

Understanding the question

The rules

```
1. MOVE   every ghost tries one step in its direction
           wall ahead?  stay put, turn 90° clockwise, +1 hit
           otherwise    step forward
2. GROUP  look at where everybody ended up
           2+ ghosts on one square?  each turns 180°, +1 hit each
3. (Part II/III) remove ghosts whose hits reached the limit
           — once after step 1, and again after step 2
```

- All ghosts move **at the same time**. Ghosts never block each other; only `#` blocks.
- Clockwise turn at a wall: `>` → `v` → `<` → `^` → `>`. The ghost does not move that round. Each ghost that bumps into a wall is one collision.
- Ghost meetings are checked **only after everyone has moved**. If $k \geq 2$ ghosts share a square, each turns around once (180°), and that's k collisions, not one per pair. A ghost can hit a wall and then also be in a meeting in the same round: 2 hits for it that round.
- Two ghosts that **swap** neighbouring squares pass through each other: they never share a square, so no collision.
- In the map pictures, a digit means "this many ghosts on this square".

Extra examples of my own (run through a copy of my Part I solution)

These are the puzzle's three small examples with a wall border added, because my solution needs one (see the warning in Part I). A "move" here is one round. The puzzle labels the *starting* picture "Round 1" in these small examples, but labels the first *result* "Round: 1" in the main example.

```
#######
#.....#
#>.#.<#
#.....#
#######
```

wall pair

```
#########
#...v...#
#..>.<..#
#.......#
#########
```

three meet

```
########
#......#
#.>..<.#
#......#
########
```

swap

Map	Collisions in moves 1, 2, 3, 4	What happened
wall pair	0, 2, 0, 2	Both step next to the middle wall (move 1), bump it and turn (move 2: `v` and `^`), walk away (move 3), then bump the outer walls (move 4).
three meet	3, 0, 1, 0	All three land on (2,4) in move 1: 3 collisions and all turn around. In move 3 the one going up bumps the top wall.
swap	0, 0, 0, 0	Move 1 brings them next to each other, move 2 swaps them, and they never share a square.

PART I Count collisions for 10 rounds

Run 10 rounds and add up all wall collisions and all ghost-meeting collisions.

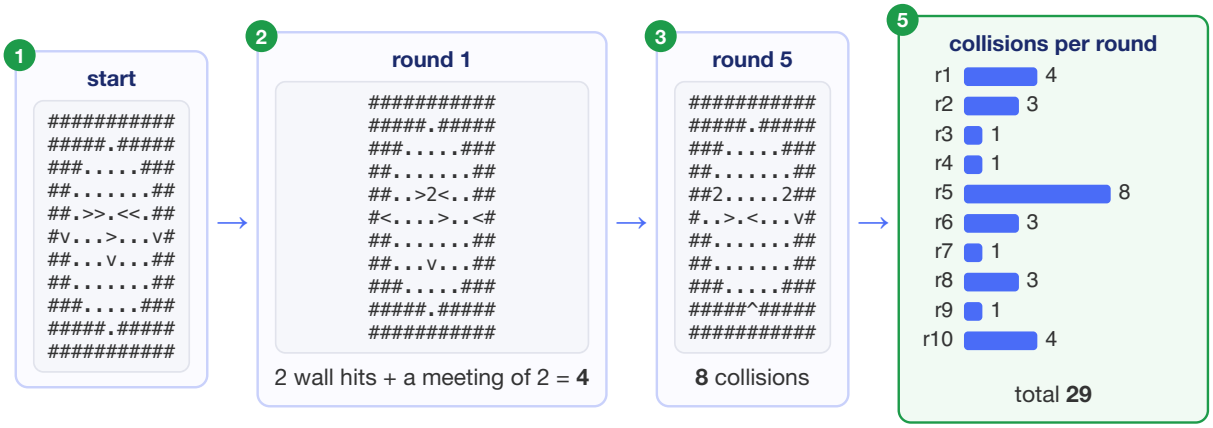
Step-by-step solution

1. Read the map as a grid.
2. Each round, scan the grid. Pick up every ghost (for a digit square, get the ghosts from last round's list), move it one step or turn it clockwise at a wall, and count wall hits.
3. Put all moved ghosts back. A square that gets a second ghost becomes `2` (+2 collisions); every further ghost adds 1.
4. Turn every ghost on a digit square around.

↻ *Repeat for 10 rounds and add up the collisions.*

How my solution works

Keep the ghosts **on the map**. Each round: pick every ghost up and move it (the new positions go into a separate list, so nobody moves twice), put them all back (a second ghost on a square makes a `2`, +2 collisions; each further one +1), then turn every ghost on a digit square around. Because a digit doesn't say which way its ghosts face, last round's list is kept too.



Dots match the step numbers above. The per-round numbers come from an instrumented copy and match the puzzle exactly.

Fragile 1: the map must have a wall all the way round. The move step looks at the square ahead without checking whether it is still on the map. On the puzzle's small example `>.#.<` written without a border, a copy of my solution ran 3 rounds and then crashed in round 4, when a ghost on the top row tried to look above it. Real chambers are always closed by `#`, so it doesn't matter for your input. Parts II and III move ghosts the same way.

Fragile 2: two copies of the truth. The ghosts live on the grid *and* in last round's list; digit squares only work because the two agree. Part II's list-only design is simpler and would also have worked here.

PART II When does the last ghost vanish?

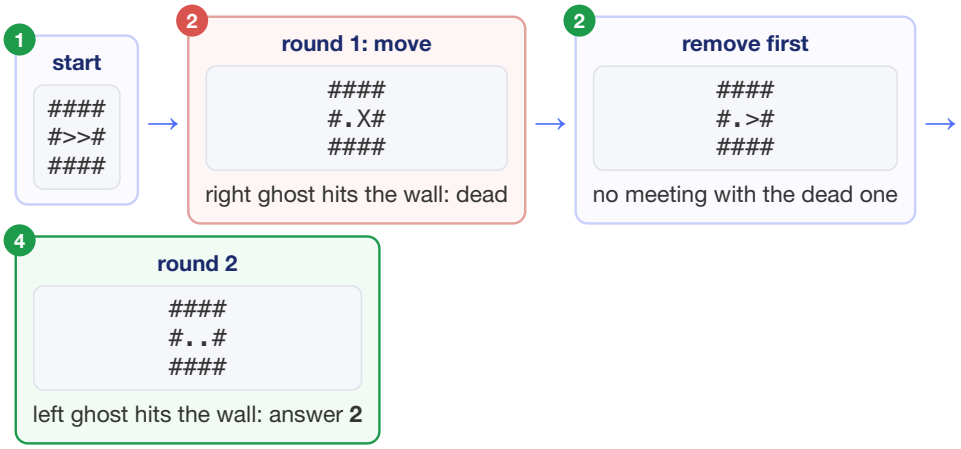
Same rules, but each ghost disappears right after its 10000th collision (10 in the example). The order matters: a ghost that dies from a wall hit is gone **before** meetings are checked, so it can't cause a meeting. Report the round in which the last ghost disappears.

Step-by-step solution

1. Take every ghost off the map into a list, with a hit counter of 0.
2. Each round: move every ghost (adding wall hits), then remove ghosts that reached the limit.
3. Group the survivors by square; in each group of 2 or more, turn each ghost around and add one hit. Remove the dead again.
4. Stop when the list is empty; the round number is the answer.

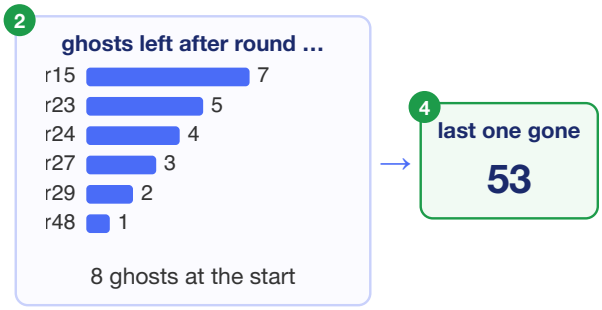
How my solution works

Take every ghost off the map into a **list** with a hit counter. Each round: move everyone, **remove** ghosts at the limit, then group the survivors by square (each ghost in a group of 2+ turns and gets one hit), and remove the dead again. Stop when the list is empty. Why removing *before* grouping matters, with a limit of 1:



Dots match the step numbers above. Grouping before removing would make them "meet" in round 1 and give 1.

On the sample, ghosts drop out like this (limit 10; with 10000 the last one goes in round **50000**):



Cost: one step per round. Fine for 50000 rounds (0.14 s on the real input), impossible for Part III's 10^{12}.

PART III A limit of a trillion

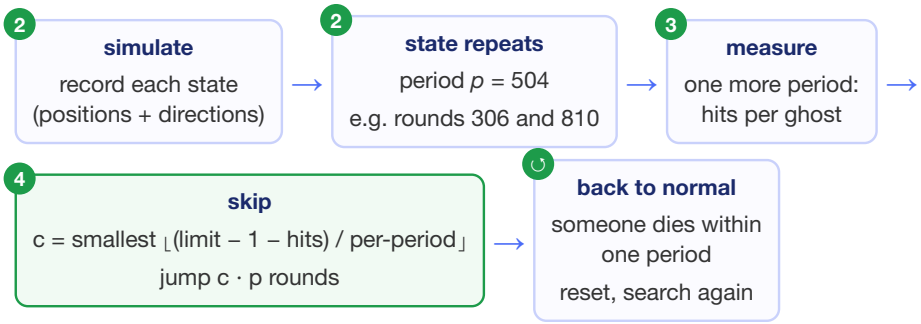
The same rules as Part II, but each ghost survives 10^{12} collisions. The example chamber has 36 ghosts and the last one disappears in round 13314695352588 (my solution gives exactly this).

Step-by-step solution

1. Run Part II's round as a step that says whether anybody died.
2. After each round with no death, record the state (positions and directions). When a state repeats, the difference in rounds is the period.
3. Simulate one more period to measure each ghost's hits per period.
4. Skip as many whole periods as possible while every ghost stays below the limit, then keep simulating normally until the next death.
5. Reset the state record after every death or skip, and repeat until no ghost is left.

How my solution works

Movement depends only on positions and directions, **not** on hits. So between deaths the state must repeat, with some period, and every ghost gains a fixed number of hits per period. Then whole periods can be skipped at once (**cycle detection**):



Dots match the step numbers above. Hits are left out of the state on purpose; a death clears the record of states.

Trace: every skip on the sample (from an instrumented copy)

Ghosts	Period	Periods skipped	Clock after the jump
36	504	1322751320	666666666594
34	504	661375659	999999999941
11	720	617283945	1444444444249
9	224	566893427	1571428573566
8	224	645628613	1716049383481
7	224	110229274	1740740741486
5	1768	52371374	1833333334580
4	7664	48904536	2208137715980
3	1768	1794677125	5381126878664
2	840	7029879709	11286225837293
1	56	36222669908	13314695352536

Eleven jumps; only 39892 rounds are actually simulated to reach round **13314695352588** (the real input: 10 jumps, about 44000 rounds). On the Part II sample the same logic gives 53 and 50000, the Part II answers.

> **Why the skip is safe:** hits only ever go up, so if every ghost is still alive at the end of the jump (hits ≤ MAX − 1), it was alive at every round in between. And the state after whole periods is the same state, so the jump lands exactly where a normal simulation would have been.
>
> **Numbers:** the largest round (about 1.3×10^{13}) is well within exact ordinary numbers.

Summary

Part	Asks for	How the ghosts are stored	Rounds simulated (sample)	Sample answer
I	collisions in 10 rounds	on the grid, plus last round's list for digit squares	10	29
II	round of the last death, limit 10 / 10000	list of ghosts with hit counters	53 / 50000	53 / 50000
III	the same, limit 10^{12}	same list, plus a record of state → round	39892	13314695352588

Key ideas: a simultaneous-move simulation is easiest with a **list** of movers and a "group by square" step after everyone has moved. When the limit is huge, notice that the state (without the counters) **repeats**, measure what one period adds, and **jump** as many periods as you safely can.